Transformée de Fourier et analyse d'un son

Projet 1

Enola Houdré Julian Discala-Porro

IPSA — Octobre 2024

Rapport remis en forme en octobre 2026, contenu et résultats inchangés

Résumé. Dans une première partie, on étudie numériquement sous Python la transformée de Fourier discrète de six signaux élémentaires. Dans une seconde partie, on analyse le spectre d'un son enregistré, on en extrait la fréquence fondamentale et les harmoniques, puis on synthétise un son à partir d'elles.

1 Présentation

Ce projet a pour objectif, dans une première partie, de comprendre et d'appréhender numériquement la transformée de Fourier avec Python, puis, dans une seconde partie, d'analyser un signal sonore à l'aide de cette transformée. On étudie les six signaux suivants :

$$x_1(t) = \sin(20\pi t), \qquad x_2(t) = \cos(20\pi t + 3), \qquad x_3(t) = 2\cos(10\pi t) + 7\cos(2\pi t),$$
$$x_4(t) = |\sin(\pi t)|, \qquad x_5(t) = \sin(50t), \qquad x_6(t) = \sin(202\pi t).$$

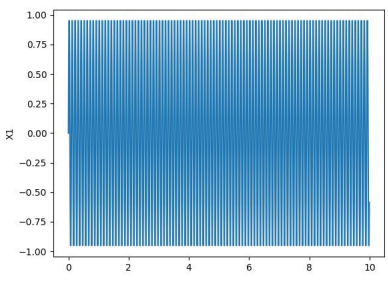
Les signaux sont échantillonnés sur $N = 1000$ points, sur une durée $T = 10\,\mathrm{s}$:

```
N = 1000
T = 10
dt = T/N
df = 1/T
t = np.linspace(0, T, N, endpoint=False)
```

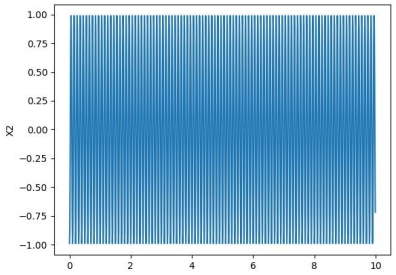
2 Partie 1 : transformée de Fourier de signaux élémentaires

2.1 Représentation temporelle

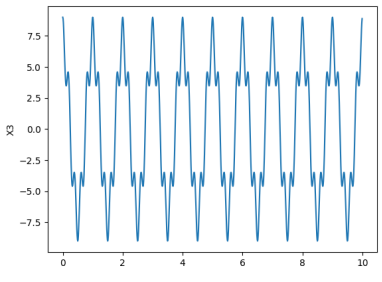
On trace d'abord les six signaux dans le domaine temporel.



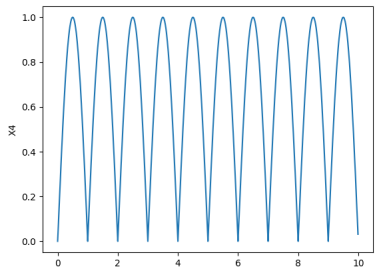
(a) x_1



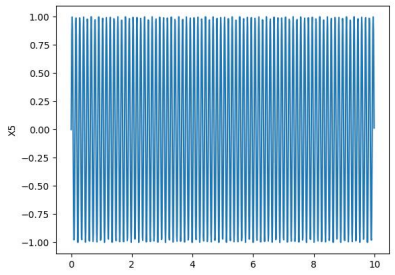
(b) x_2



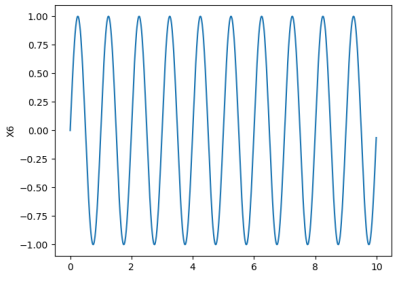
(c) x_3



(a) x_4



(b) x_5

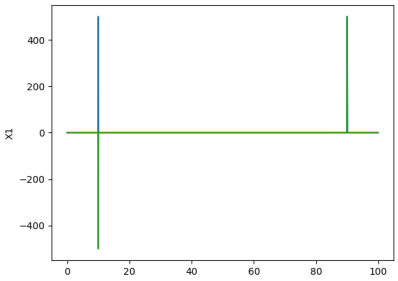


(c) x_6

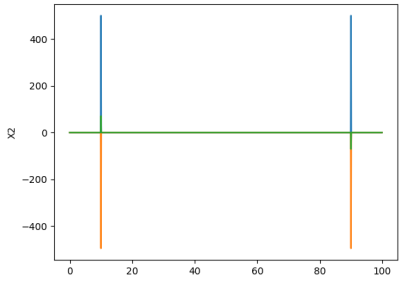
2.2 Représentation fréquentielle

On calcule ensuite la transformée de Fourier discrète de chaque signal avec `np.fft.fft`, et on trace son module (en bleu), sa partie réelle (en orange) et sa partie imaginaire (en vert) :

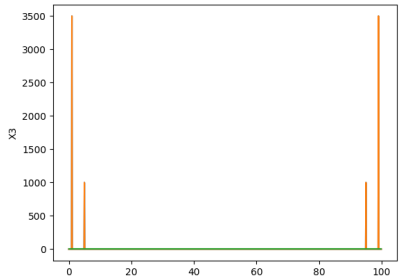
```
X1 = np.fft.fft(x1)
f = np.linspace(0, N*df, N, endpoint=False)
plt.plot(f, np.abs(X1))
plt.plot(f, np.real(X1))
plt.plot(f, np.imag(X1))
```



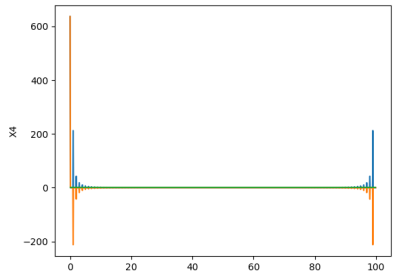
(a) X_1



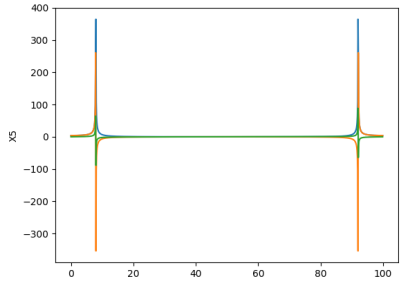
(b) X_2



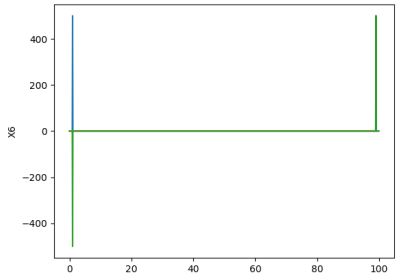
(c) X_3



(a) X_4



(b) X_5



(c) X_6

2.3 Interprétation des spectres

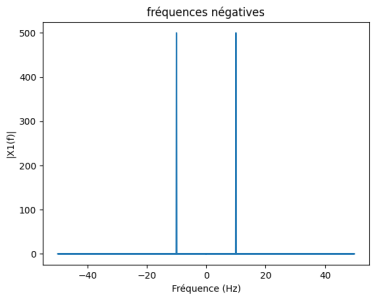
Le spectre est nul presque partout, sauf à certaines fréquences où apparaît un pic de valeur bien plus élevée. Ce sont ces pics qui permettent d'identifier les fréquences des sinusoïdes qui composent chaque signal.

2.4 Fréquences négatives

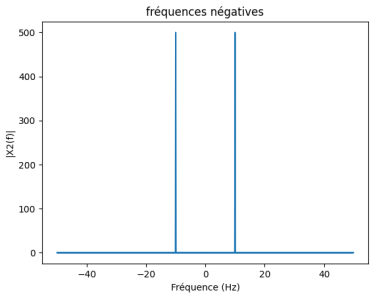
Les fréquences obtenues dans l'intervalle $f_e/2 < f_k < f_e$ peuvent être interprétées comme des fréquences négatives, grâce à la symétrie de la transformée de Fourier. On le voit bien

graphiquement : chaque signal présente un pic à gauche et un autre à droite, en $f_k - f_e$.

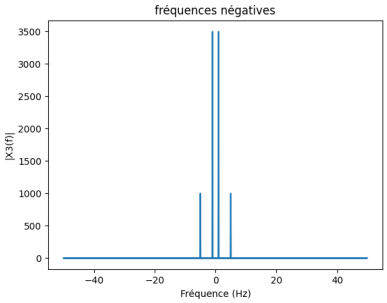
En adoptant ce point de vue, c'est-à-dire en centrant l'axe des fréquences sur zéro avec `np.fft.fftfreq`, on obtient les spectres suivants :



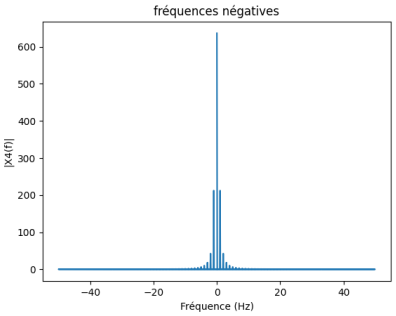
(a) $|X_1|$



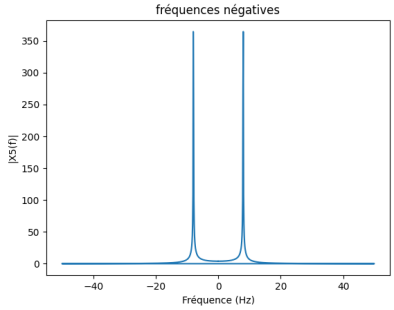
(b) $|X_2|$



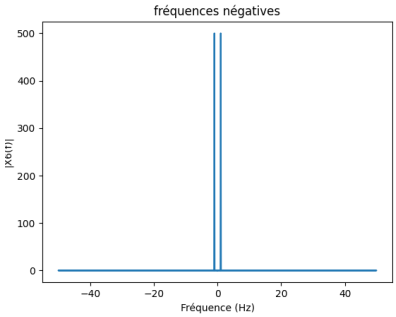
(c) $|X_3|$



(a) $|X_4|$



(b) $|X_5|$



(c) $|X_6|$

2.5 Cas d'un signal réel

Lorsque $x(t)$ est réel, on observe graphiquement la relation

$$X(-f) = \overline{X(f)}. \tag{1}$$

Les parties réelles sont égales et les parties imaginaires opposées : $\operatorname{Im} X(-f) = -\operatorname{Im} X(f)$. Autrement dit, $X(-f)$ est le conjugué de $X(f)$.

3 Partie 2 : analyse du spectre d'un fichier audio

3.1 Lecture du fichier

On lit le fichier audio avec la fonction `scipy.io.wavfile.read`, qui renvoie la fréquence d'échantillonnage f_e et les échantillons dans un tableau :

```
fe, signal = wavfile.read('Note_mono_08.wav')
```

3.2 Durée du signal

La durée du fichier, en millisecondes, se déduit du nombre d'échantillons N et de la fréquence d'échantillonnage :

$$\text{durée} = \frac{N}{f_e} \times 1000. \tag{2}$$

Avec $N = 101\,999$, on obtient une durée de 2312,90 ms.

3.3 Représentation temporelle

On trace ensuite l'amplitude du signal en fonction du temps.

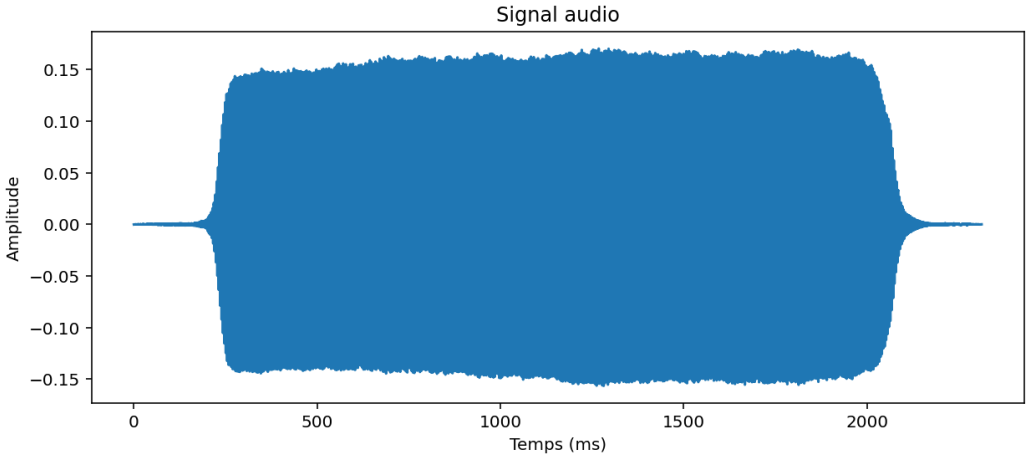


Figure 7. Signal audio en fonction du temps.

3.4 Représentation fréquentielle

Une transformée de Fourier donne la représentation fréquentielle du signal et permet de visualiser les différentes fréquences présentes dans le son. On ne conserve que les fréquences positives :

```
transfo_fourier = fft(signal)
frequences = np.fft.fftfreq(N, 1/fe)
positive_freq = frequences[:N//2]
positive_spectrum = np.abs(transfo_fourier[:N//2])
```

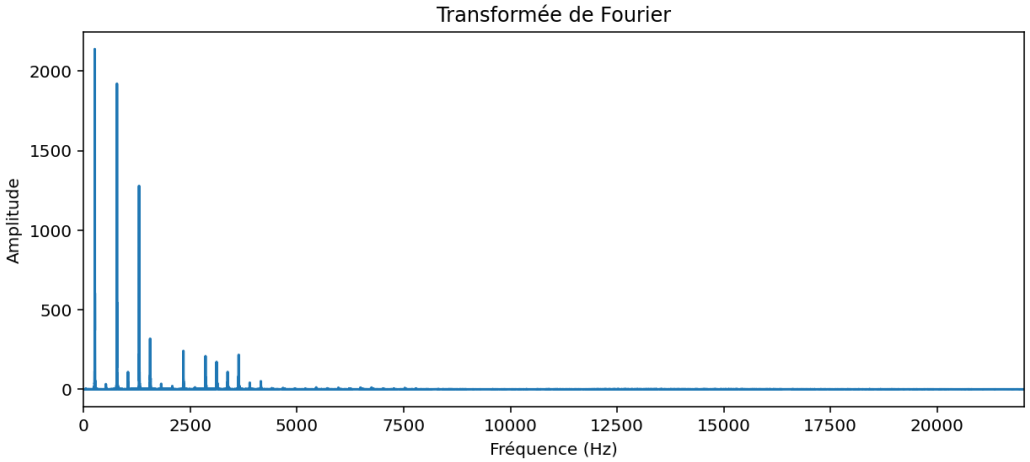


Figure 8. Spectre du signal audio.

3.5 Fréquence fondamentale et harmoniques

On identifie la fréquence fondamentale F_0, au maximum du spectre, puis les premières harmoniques par la méthode des pics.

Harmonique	Fréquence (Hz)	Amplitude
1	259,41	2138,77
2	518,83	32,89
3	778,24	1495,45
4	1037,66	63,52
5	1297,07	616,56

Table 1. Fréquence fondamentale et premières harmoniques du son analysé.

3.6 Synthèse d'un son

Enfin, on crée un fichier audio synthétique en additionnant des sinusoïdes aux fréquences de la fondamentale et de ses harmoniques, pondérées par leurs amplitudes :

```
duree_synth = 3
t = np.linspace(0, duree_synth, int(fe*duree_synth), endpoint=False)
signal_synthetique = np.zeros_like(t)
for f, A in zip(harmoniques, amplitudes):
    signal_synthetique += A*np.cos(2*np.pi*f*t)
signal_synthetique /= np.max(np.abs(signal_synthetique))
sf.write('Mon_son.wav', signal_synthetique, fe)
```